

Rapport Technique & Plan d'Exécution Global

Plateforme E-commerce « Le Laboratoire » & Maximisation de la Fayda
Architecture Découplée, Pipeline IA 24/7 & Écosystème 100% Gratuit (0 DA)

Équipe d'Ingénierie Logicielle & Architecture IA

18 septembre 2026

Résumé

Ce rapport formalise l'analyse détaillée du cahier des charges de la plateforme e-commerce « **Le Laboratoire** ». L'application a pour mission stratégique d'évaluer la rentabilité d'une marchandise (*sel3a*) sur un cycle strict de 6 jours avec un capital réduit avant investissement de gros, tout en optimisant le bénéfice net (*Fayda*). Ce document établit la sélection rigoureuse des technologies de A à Z (garantissant **0 DA** de charges récurrentes), détaille la configuration d'un module d'intelligence artificielle automatisé 24/7 en mode cloud sans machine locale, intègre un scraping non bloquant via DuckDuckGo API, et déroule une feuille de route méthodologique par phases successives pour une mise en production rapide et sécurisée.

Table des matières

1	Analyse Stratégique et Modélisation Métier	2
1.1	Problématique E-commerce en Algérie	2
1.2	Formulation Mathématique de la Fayda	2
1.3	Contraintes Critères et Objectifs Opérationnels	2
2	Cartographie Technologique de A à Z (Stack 100% Gratuite)	3
2.1	Frontend : React 19, Vite et Spécifications PWA Mobile-First	3
2.2	Backend : C# ASP.NET Core Web API & Entity Framework Core	4
2.3	Gestion Volatile des Images via Cloudinary	4
2.4	Module IA 24/7 : Groq Cloud API & Scraping DuckDuckGo	4
3	Modèle de Données et Architecture Relationnelle	5
4	Logique d'Arbitrage et Feedback des 6 Jours	5
4.1	Arrondi Psychologique Algérien	5
4.2	Les Trois Sentinelles d'Arbitrage (Boucle de 6 Jours)	5
5	Plan de Développement Détaillé de A à Z	6
5.1	Phase 0 : Initialisation et Préparation de l'Écosystème (Jour 1)	6
5.2	Phase 1 : Architecture Backend .NET & Base de Données (Jours 2 à 4)	6
5.3	Phase 2 : Module Médias & Contrôleurs Métier (Jours 5 à 6)	7
5.4	Phase 3 : Interface Utilisateur PWA Mobile-First (Jours 7 à 10)	7
5.5	Phase 4 : Pipeline IA 24/7 (DuckDuckGo + Groq API) (Jours 11 à 13)	7
5.6	Phase 5 : Déploiement Cloud 0 DA & Intégration Continue (Jours 14 à 15)	7
6	Conclusion et Recommandations Opérationnelles	8

1 Analyse Stratégique et Modélisation Métier

1.1 Problématique E-commerce en Algérie

Le commerce électronique en Algérie (modèle de distribution locale, achat sur les marchés de gros d'El Eulma, Bab Ezzouar, ou import direct) expose les entrepreneurs à un risque de trésorerie majeur : l'acquisition aveugle d'importants volumes sans certitude de conversion sur les réseaux sociaux.

Le projet « **Le Laboratoire** » répond à cette friction par un environnement d'arbitrage contrôlé :

1. **Achat d'un échantillon test** : Acquisition d'un stock minimal (ex. 8 pièces) définissant le **Ras Lmal**.
2. **Test Publicitaire Sponsoring (Ads)** : Lancement d'une campagne Meta (Facebook/Instagram) avec un budget d'amorçage standardisé (2 000 DA alloués sur 6 jours).
3. **Logistique Réelle** : Intégration du ticket de bureau (frais d'expédition et de bordereau par colis livré, défaut : 15 DA).
4. **Calcul Déterministe de la Fayda** : Évaluation en temps réel du seuil de rentabilité et du retour net sur capital investi.

1.2 Formulation Mathématique de la Fayda

Le backend .NET prend en charge les équations fondamentales sans approximation en arithmétique décimale haute précision (**decimal**) :

$$\text{Dépense Totale} = \text{Ras Lmal} + \sum_{j=1}^n \text{Ads Consommé}_j + (\text{Ventes Confirmées} \times \text{Ticket Bureau}) \quad (1)$$

$$\text{Revenu Total} = \text{Ventes Confirmées} \times \text{Prix de Vente Unitaire Effectif} \quad (2)$$

$$\textbf{Fayda} = \text{Revenu Total} - \text{Dépense Totale} \quad (3)$$

$$\textbf{ROI} = \left(\frac{\textbf{Fayda}}{\text{Dépense Totale}} \right) \times 100\% \quad (4)$$

1.3 Contraintes Critères et Objectifs Opérationnels

- **Coût d'exploitation = 0 DA** : Toutes les couches architecturales (Frontend, Backend, Base de données, Storage d'images, Scraping, Inférence LLM) doivent exploiter des tiers gratuits stables et pérennes.
- **Exploitation Mobile-First** : L'un des deux associés opère exclusivement sur smartphone en déplacement. L'interface doit fonctionner sous forme de Progressive Web App (PWA), s'installer sur l'écran d'accueil et maximiser la rapidité d'entrée des métriques.
- **Résilience du Stockage** : L'hébergement gratuit de conteneurs (Render/Koyeb) implique un système de fichiers volatil (*ephemeral file system*). Les images ne doivent en aucun cas résider sur le serveur sous peine d'être anéanties au premier redémarrage.
- **Sécurité Contrôlée** : Système restreint à deux administrateurs authentifiés par JSON Web Token (JWT). Aucun accès d'enregistrement (*self-register*) n'est exposé.

2 Cartographie Technologique de A à Z (Stack 100% Gratuite)

Couche	Technologie	Rôle & Justification Technique	Coût
Frontend	React 19 + Vite.js	PWA ultra-rapide, rendu instantané, gestion fluide de l'état.	0 DA
Styling	Tailwind / Vanilla CSS	Mobile-First, Bottom Navigation, pavé tactile adapté.	0 DA
PWA Engine	Vite Plugin PWA	Service Worker, Manifest JSON, cache applicatif, icônes natives.	0 DA
Backend	ASP.NET Core 10/9	Web API typée, haute performance, calcul financier déterministe.	0 DA
ORM	EF Core + Npgsql	Mapping objet-relationnel, migrations de schéma, support jsonb.	0 DA
Base Données	PostgreSQL (Neon.tech)	0.5 Go gratuit, SSL natif, stockage relationnel persistant.	0 DA
Médias	Cloudinary API	Hébergement cloud persistant, compression WebP/AVIF auto.	0 DA
Scraping	DuckDuckGo Search API	Dorking non bloquant, sans token, extraction gratuite de snippets.	0 DA
Moteur IA	Groq Cloud API	Inférence LLM 24/7 (DeepSeek-R1 / LLaMA-3.3 70B), latence < 1s.	0 DA
Déploiement Front	Vercel	Hébergement PWA, CDN mondial, HTTPS géré automatiquement.	0 DA
Déploiement Back	Render (Docker)	Conteneur Linux ASP.NET Core isolé, CI/CD depuis Git.	0 DA
Anti-Veille	Cron-Job.org	Requête HTTP /health toutes les 14 min contre le cold-start.	0 DA

TABLE 1 – Matrice d'ingénierie logicielle et allocation des services gratuits (0 DA)

2.1 Frontend : React 19, Vite et Spécifications PWA Mobile-First

Le frontend est architecturé pour reproduire la sensation tactile d'une application native Android/iOS :

- **Bottom Navigation Bar** : Accès permanent à portée de pouce aux 4 espaces : *Laboratoire*, *Produits*, *Fayda Dashboard*, *Paramètres*.
- **Fast-Input Numérique** : Utilisation systématique de la directive HTML5 `inputMode="decimal"` sur les champs financiers (dépense Ads, prix de vente, volume). Ceci force l'affichage immédiat du pavé numérique natif du smartphone, évitant les erreurs de saisie.
- **Capture Photo Directe** : Le composant d'upload produit emploie l'attribut standard :

```
<input type="file" accept="image/*" capture="environment" />
```

Cette balise active sans intermédiaire l'objectif photo dorsal du téléphone pour photographier directement la marchandise reçue.

- **Offline Shell** : Mise en cache des composants d'interface via Workbox pour charger l'application instantanément même en connexion 3G instable.

2.2 Backend : C# ASP.NET Core Web API & Entity Framework Core

Le framework .NET offre une rigueur mathématique et une vitesse d'exécution incomparables :

- **Précision Financière** : Toutes les colonnes de montants utilisent le type `decimal` (128 bits, base 10), garantissant l'absence totale de dérive d'arrondi binaire inhérente aux types `float` ou `double`.
- **Stockage Semi-Structuré (JSONB)** : La table des produits intègre une colonne `Specifications` typée en `jsonb` PostgreSQL via EF Core. Cela autorise la saisie dynamique d'attributs arbitraires (variantes de couleurs, calibres, pointures, matière) sans exiger de migrations de schéma SQL répétitives.
- **Authentification Restreinte** : Émission de jetons JWT cryptographiques signés en HMAC-SHA256. La base est initialisée avec exactement 2 comptes gérants hachés en BCrypt. Aucun point de terminaison d'inscription n'est exposé.

2.3 Gestion Volatile des Images via Cloudinary

Pour contourner la volatilité du disque local de Render, le flux de capture d'image adopte une stratégie d'upload direct :

1. L'administrateur prend la photo dans la PWA.
2. Le client web transmet le fichier binaire directement vers l'API Cloudinary au moyen d'un *Unsigned Upload Preset* dédié et restreint.
3. Cloudinary effectue les transformations automatiques (conversion automatique au format WebP, dimensionnement responsive, compression sans perte perceptible).
4. Cloudinary retourne une URL sécurisée pérenne (ex. `https://res.cloudinary.com/.../sel3a_894.webp`).
5. Seule cette chaîne textuelle est communiquée au backend .NET pour stockage dans PostgreSQL.
6. **Résultat** : 0 Mo d'espace disque consommé sur le serveur backend, persistance garantie à 100% lors des cycles de sommeil du conteneur.

2.4 Module IA 24/7 : Groq Cloud API & Scraping DuckDuckGo

En accord avec le choix de l'**Option B**, le sous-système d'intelligence artificielle est déporté dans le cloud avec une disponibilité 24h/24 sans nécessiter de machine locale :

- **Moteur d'inférence LLM (Groq)** : Utilisation du modèle **DeepSeek-R1-Distill-Llama-70B** ou **LLaMA-3.3-70B-Versatile** via l'API gratuite de Groq. Groq offre un débit ultra-performant (> 250 tokens/seconde) sous quota généreux gratuit, assurant un temps de réponse inférieur à 1,5 seconde.
- **Scraping via DuckDuckGo API** : Utilisation de la bibliothèque Python `duckduckgo-search`.
 - **Avantages** : Totalelement gratuit, aucune clé d'API requise, aucun blocage agressif de Captcha, pas de quota mensuel contraignant comme SerpApi.
 - **Requête automatisée** :

```
"Nom de la sel3a" (DA OR DZD) (site:facebook.com OR site:instagram.com OR site:tiktok.com) Algérie
```
- **Extraction et Normalisation Sémantique** : Le LLM filtre le bruit textuel des snippets renvoyés par DuckDuckGo, isole les prix réels en Dinars Algériens, élimine les valeurs aberrantes (frais de port ou prix grossistes), et produit un document JSON normalisé.

3 Modèle de Données et Architecture Relationnelle

Le schéma relationnel est conçu pour assurer l'intégrité référentielle et la traçabilité des métriques journalières :

Table	Clé / Index	Colonnes & Types
Users	Id (PK, Guid)	Username (VarChar 50, Unique), PasswordHash (Text), Role (VarChar 20), CreatedAt (Timestamp)
Products	Id (PK, Guid)	Name (VarChar 150), Category (VarChar 50), ImageUrl (Text), Specifications (JSONB), BuyPrice (Decimal), TargetSellPrice (Decimal), CreatedAt (Timestamp)
ProductTests	Id (PK, Guid) ProductId (FK)	InitialQuantity (Int), RemainingQuantity (Int), RasLmal (Decimal), AdsBudgetTotal (Decimal), TicketBureauFee (Decimal), SellingPrice (Decimal), AIRecommendedPrice (Decimal), Status (VarChar 20), StartDate (Date), EndDate (Date)
TestDailyMetrics	Id (PK, Guid) ProductTestId (FK)	DayNumber (Int, 1..6), MetricDate (Date), AdsSpent (Decimal), Impressions (Int), Clicks (Int), ConfirmedOrders (Int), TotalRevenue (Decimal), NetFayda (Decimal), Notes (Text)
AiMarketInsights	Id (PK, Guid) ProductId (FK)	MinObservedPrice (Decimal), MaxObservedPrice (Decimal), AveragePrice (Decimal), PsychologicalPrice (Decimal), RawJsonData (JSONB), AnalyzedAt (Timestamp)

FIGURE 1 – Spécification du dictionnaire de données PostgreSQL

4 Logique d'Arbitrage et Feedback des 6 Jours

4.1 Arrondi Psychologique Algérien

Les consommateurs sur les réseaux sociaux réagissent fortement aux seuils psychologiques. Le système intègre un algorithme d'ajustement automatique :

- Un prix brut calculé de 3 000 DA est automatiquement repositionné à **2 900 DA** ou **2 990 DA**.
- Un prix intermédiaire de 4 100 DA est repositionné à **3 900 DA** pour franchir la barrière des 4 000 DA.

4.2 Les Trois Sentinelles d'Arbitrage (Boucle de 6 Jours)

Durant les 6 jours de validation d'un test, le moteur d'analyse compare chaque soir les données consolidées :

1. Alerte 1 : « Prix Trop Élevé » (Jour 3) :

$$\text{Condition : } (\text{Jour} \geq 3) \wedge (\text{CTR} \geq 2.0\%) \wedge (\text{Ventes Confirmées} = 0) \quad (5)$$

Diagnostic : L'annonce suscite l'intérêt (bon visuel, clics nombreux), mais l'acte d'achat est bloqué sur la page d'atterrissage. Le prix de vente est supérieur au seuil d'acceptation du marché.

Action système : Notification prioritaire recommandant une décote immédiate de 10% à 15% sur le prix affiché.

2. Alerte 2 : « Opportunité de Maximisation » :

Condition : $(\text{Stock Restant} \leq 25\%) \wedge (\text{Jour} \leq 4) \wedge (\text{CPA} < 0.20 \times \text{Prix Vente})$ (6)

Diagnostic : Forte vélocité d'écoulement du stock avec un coût par acquisition publicitaire très bas.

Action système : Recommandation d'augmentation du prix unitaire sur le stock résiduel pour maximiser la marge nette (*Fayda*) sans impacter le volume de clôture.

3. Alerte 3 : « Arrêt d'Urgence (Cut-Loss) » :

Condition : $(\text{Ads Consommé} \geq 0.60 \times \text{Budget Total}) \wedge (\text{Ventes Confirmées} = 0)$ (7)

Diagnostic : 60% du budget sponsoring (soit $\geq 1\,200$ DA sur les 2 000 DA prévus) a été dépensé sans générer la moindre conversion.

Action système : Alerte rouge critique avec recommandation impérative de stopper immédiatement la campagne publicitaire Meta afin de préserver les 800 DA restants du budget.

5 Plan de Développement Détaillé de A à Z

5.1 Phase 0 : Initialisation et Préparation de l'Écosystème (Jour 1)

- Création de l'arborescence du projet (`/frontend`, `/backend`, `/ai-worker`).
- Souscription des comptes gratuits :
 - Instance PostgreSQL sur **Neon.tech** (collecte de l'URI de connexion SSL).
 - Espace de stockage sur **Cloudinary** (création du Cloud Name et du Preset non signé).
 - Clé d'API gratuite sur la console **Groq Cloud**.
- Configuration du dépôt Git avec gestion fine des variables d'environnement confidentielles (`.env`, `appsettings.Production.json` dans le `.gitignore`).

5.2 Phase 1 : Architecture Backend .NET & Base de Données (Jours 2 à 4)

- Création du projet `dotnet new webapi -n LeLaboratoire.Api`.
- Implémentation des entités de domaine (`User`, `Product`, `ProductTest`, `TestDailyMetric`).
- Configuration d'`AppDbContext` avec le provider `Npgsql` et mapping de la propriété `Specifications` en `jsonb`.
- Mise en place du module de sécurité :
 - Seeder initial créant les 2 comptes administrateurs avec hachage `BCrypt`.
 - Contrôleur d'authentification `POST /api/auth/login` avec génération de token JWT sécurisé.
 - Middleware d'autorisation restreignant l'accès aux endpoints protégés.
- Écriture des tests unitaires sur le moteur de calcul financier `FaydaCalculatorService`.

5.3 Phase 2 : Module Médias & Contrôleurs Métier (Jours 5 à 6)

- Implémentation du contrôleur **ProductsController** (CRUD des marchandises, intégration de la référence d'image Cloudinary).
- Implémentation du contrôleur **TestsController** :
 - Démarrage d'un test 6 jours (saisie du *Ras Lmal*, stock initial, budget Ads).
 - Enregistrement des métriques journalières (`POST /api/tests/{id}/metrics/day-{n}`).
 - Clôture et archivage du test avec calcul du ROI final.

5.4 Phase 3 : Interface Utilisateur PWA Mobile-First (Jours 7 à 10)

- Initialisation du frontend : `npm create vite@latest frontend - -template react`.
- Installation et configuration de `vite-plugin-pwa` :
 - Définition du fichier `manifest.json` avec mode `standalone`.
 - Enregistrement du Service Worker pour l'installation sur Android/iOS.
- Développement des vues maîtresses :
 - **BottomBar** permanente avec icônes Lucide adaptées au smartphone.
 - **Formulaire Fast-Input** avec pavé numérique forcé (`inputMode="decimal"`).
 - **Composant Caméra Terrain** déclenchant la caméra arrière et upload direct sur Cloudinary.
 - **Dashboard Laboratoire** avec progression visuelle des 6 jours et indicateurs couleur de la Fayda.

5.5 Phase 4 : Pipeline IA 24/7 (DuckDuckGo + Groq API) (Jours 11 à 13)

- Création du service Python autonome ou script exécutable :
 - Intégration de `duckduckgo-search` pour l'exécution des Google/DDG Dorks.
 - Construction des requêtes ciblées sur les réseaux sociaux en Algérie.
- Intégration du client Groq API :
 - Élaboration du System Prompt contraignant le LLM à délivrer un schéma JSON strict.
 - Calcul de l'arrondi psychologique et du prix plafond suggéré.
- Connexion avec le backend .NET via endpoint webhook ou intégration directe C# HttpClient appelant Groq si l'on souhaite un backend unifié sans serveur Python séparé.

5.6 Phase 5 : Déploiement Cloud 0 DA & Intégration Continue (Jours 14 à 15)

- **Dockerisation** : Rédaction d'un `Dockerfile` multi-stage optimisé pour le backend .NET (Linux Alpine).
- **Déploiement Backend** : Connexion du dépôt Git à **Render.com** (plan Web Service Free).
- **Déploiement Frontend** : Connexion à **Vercel** avec configuration de réécriture pour SPA (`vercel.json`).
- **Configuration Keep-Alive** : Création d'une tâche récurrente sur **Cron-Job.org** envoyant une requête `GET /api/health` toutes les 14 minutes afin d'éliminer la latence de réveil du serveur gratuit.
- **Recette Terrain** : Validation physique sur smartphone des flux d'upload photo, de saisie de métriques et de navigation hors-ligne.

6 Conclusion et Recommandations Opérationnelles

Cette architecture respecte l'intégralité des exigences techniques, financières et fonctionnelles du cahier des charges :

- **Absence de coût récurrent** : La synergie entre Neon, Cloudinary, Vercel, Render, Groq et DuckDuckGo garantit une exploitation à 0 DA de budget d'infrastructure.
- **Efficacité opérationnelle** : La PWA supprime la friction de saisie sur le terrain pour l'administrateur nomade.
- **Sécurisation du capital** : Les 3 règles d'arbitrage automatique protègent le *Ras Lmal* contre les campagnes publicitaires non rentables.

L'équipe d'ingénierie est prête à lancer la Phase 0 dès validation de ce rapport.